

Calcolo Parallelo e Distribuito

Decomposizione dati 2D:
Algoritmi full-parallel per la gestione di matrici
esercitazione

Docente: Prof. L. Marcellino
Tutor: Prof. P. De Luca

… lo vediamo lunedì

INFORMAZIONI IMPORTANTI PER L'ESAME:

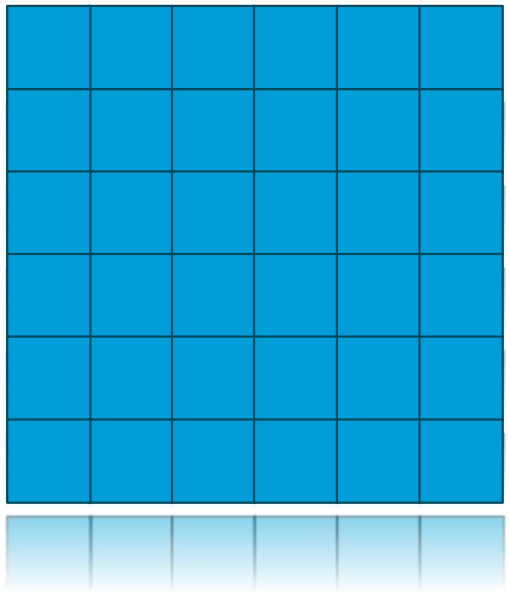
- **PROVA scritta: durata 2 ore**

Descrizione di un algoritmo parallelo relativo ad un problema assegnato, Calcolo Teorico di una o più metriche di valutazione (Speedup ed Efficienza, Isoefficienza, Ware-Amdhal), **decrizione e commento del codice relativo all'algoritmo**

- **PROVA orale (obbligatoria), ma solo se ammessi (scritto sufficiente)**

Decomposizione di matrici

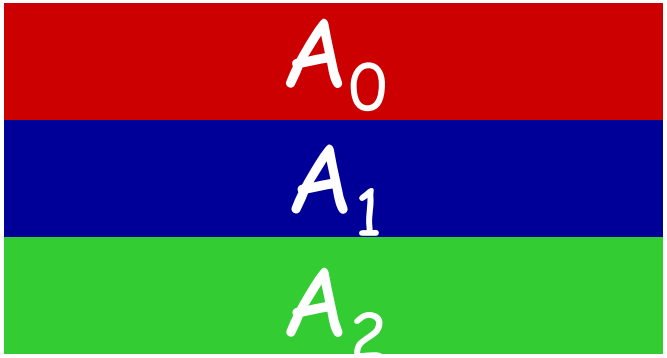
Prodotto di uno scalare con una matrice di grandi dimensioni!

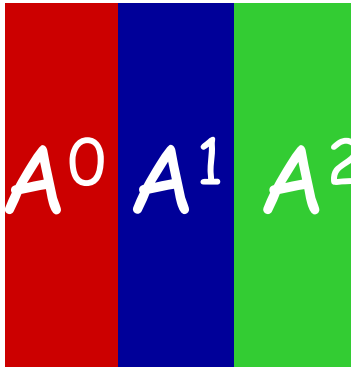


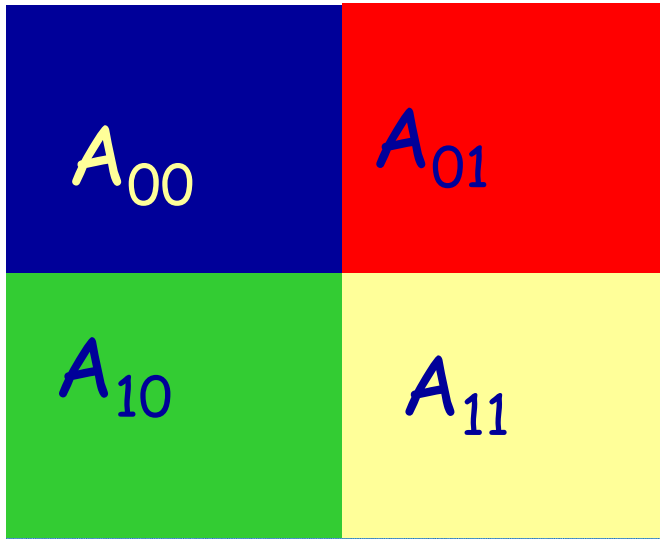
Input: $\beta \cdot A$: $\dim(A)=N \times N$

Output: $C=\{c_{i,j}\} = \{\beta \cdot a_{i,j}\}$ $i=0,\ldots,N-1$, $j=0,\ldots,N-1$

Cosa succede se le matrici sono del tipo N ×M



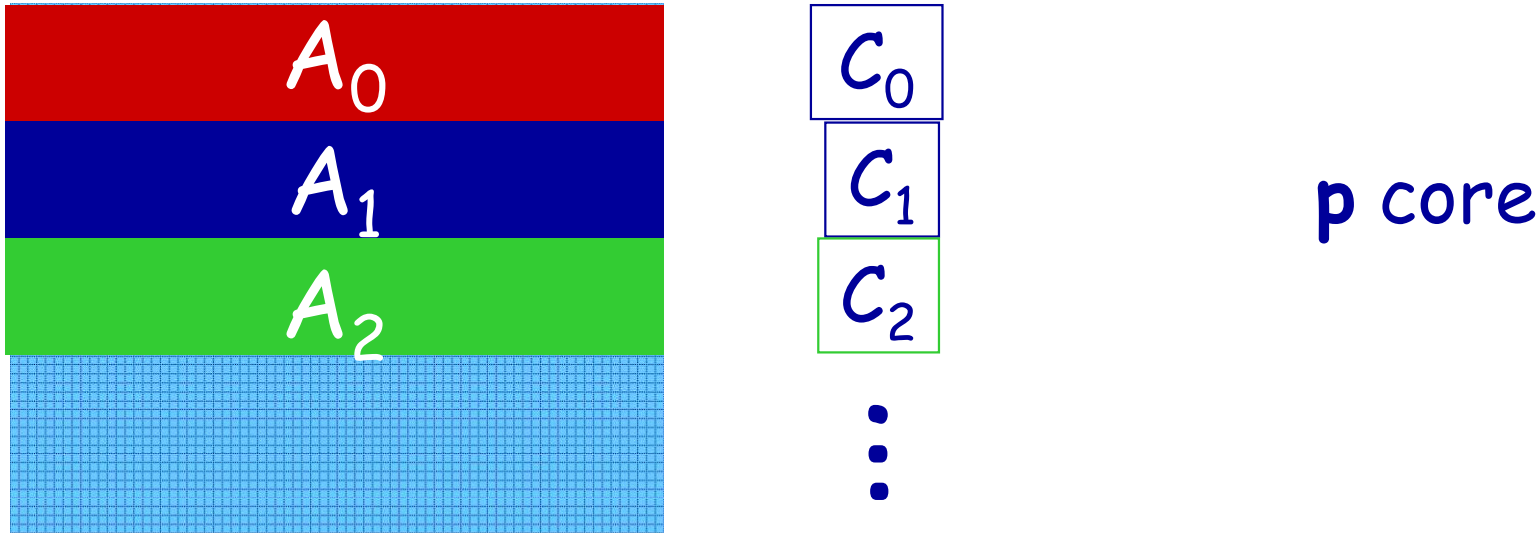




I STRATEGIA

Suddividiamo la matrice A in

BLOCCHI di RIGHE



prodotto di uno scalare
per una matrice

$$\dim[A] = N \times M$$

In sequenziale

$$T_1(N \times M) = N \times M$$

In Parallelo np core

1 strategia – np=p righe

$$S_p = T_1(N \times M)/T_p(N \times M) = N \times M / [(N/p) \times M] = p$$

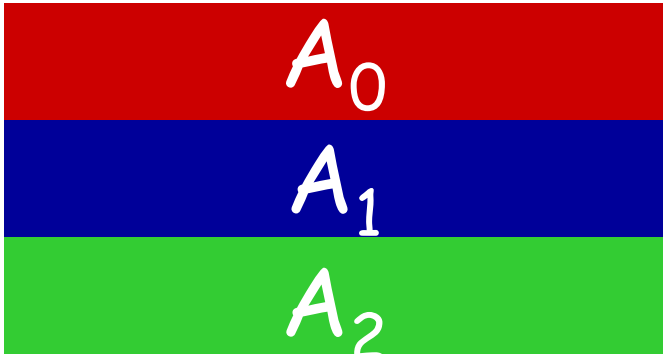
2 strategia – np=q colonne

$$S_q = T_1(N \times M)/T_q(N \times M) = N \times M / [N \times (M/q)] = q$$

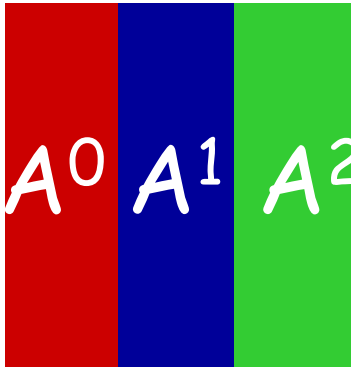
3 strategia – np=p×q righe-colonne

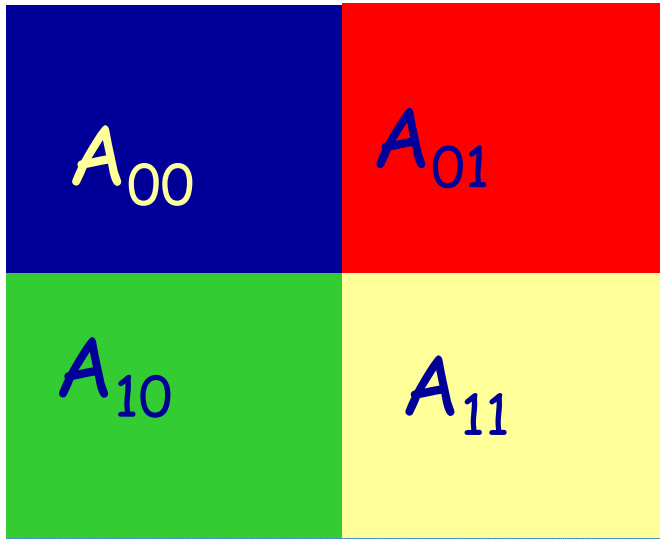
$$S_{pq} = T_1(N \times M)/T_{pq}(N \times M) = N \times M / [(N/p) \times (M/q)] = pq$$

Cosa succede se le matrici sono del tipo N ×M e le dimensioni non sono esattamente divisibili per p o q ???



…





Esercitazione in aula

Cosa succede se le matrici sono del tipo N ×M e le dimensioni non sono
esattamente divisibili per p o q ???

Nucleo: Prodotto scalare per Matrice – 1 strategia

```
#pragma omp parallel for shared(m,n,A, B, alpha) private(i,j)
for (i=0; i<n; i++){
        for (j=0; j<m; j++)
                B[i][j] = A[i][j]*alpha;
}
```

Nucleo: Prodotto scalare per Matrice – 2 strategia

```
#pragma omp parallel for shared(m,n,A, B, alpha) private(i,j)
for (j=0; j<m; j++){
        for (i=0; i<n; i++)
                B[i][j] = A[i][j]*alpha;
}
```

Nucleo: Prodotto scalare per Matrice 3 strategia

Provate da soli…

N numero righe - M numero di colonne

nthreads = omp_get_max_threads();

grid_rows, grid_cols // attenzione!

// Calcolo dimensioni dei blocchi – Attenzione all'esatta divisibilità
block_rows = N / grid_rows;
block_cols = M / grid_cols;

Nucleo: Prodotto scalare per Matrice 3 strategia

Con la clausola collapse (2) [due for innestati] è possibile espandere espandere il costrutto parallel for anche al secondo for

```
% grid_rows=p. grid_cols=q
// Parallelizzazione a blocchi
    #pragma omp parallel for collapse(2) % tratta tutto come un unico ciclo
    for (bi = 0; bi < grid_rows; bi++) {
        for (bj = 0; bj < grid_cols; bj++) {
            row_start = bi * block_rows;
            row_end = (row_start + block_rows > N) ? N : row_start + block_rows;
            col_start = bj * block_cols;
            col_end = (col_start + block_cols > M) ? M : col_start + block_cols;

            for (int i = row_start; i < row_end; i++) {
                for (int j = col_start; j < col_end; j++) {
                    matrix[i][j] *= alpha;
                }
            }
        }
    }
```

Cosa propone l’AI?